

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi:10.18637/jss.v000.i00

unitTransformDist: A Distribution-Agnostic R Package for Unit-Interval Models via the Odds-Inverting Transform

Mohamad Yamen Al Mohamad
University of Tabriz

Ali Akbar Heydari
University of Tabriz

Hossein Jabbari Khamnei
University of Tabriz

Abstract

Data restricted to the open interval $(0, 1)$ – proportions, rates, and probabilities – are commonly modelled in R with **betareg** (beta regression), **simplexreg** (simplex regression), or the recently released **gkwdist** (generalized Kumaraswamy family). Each of these packages targets one specific parametric family, fixed in advance by its authors. This paper introduces **unitTransformDist**, which instead constructs a $(0, 1)$-valued distribution from an arbitrary base distribution on $(0, \infty)$ via the transform $Y = 1/(1 + X)$, taking the base distribution's density, cumulative distribution function, quantile function, and random-generation function directly as arguments rather than selecting from a fixed registry. This design lets users plug in R's built-in distributions or their own custom research distributions – including ones with no closed-form quantile function – without modifying any package code. We derive the transform's density, distribution, quantile, moment, and hazard relationships in general form, establish an invariance property of the likelihood under the transform that underlies every inferential routine in the package, and use it to document and empirically correct a subtle but consequential implementation error common to profile-likelihood confidence intervals: freezing nuisance parameters at their unconstrained maximum-likelihood estimate while profiling a parameter of interest, rather than re-optimizing them at each candidate value. A reproducible Monte Carlo study included in the package shows this mistake holds actual 95% coverage near 54–61% across sample sizes $n \in \{30, 60, 100, 200\}$ for a two-parameter gamma base distribution, essentially independent of n, while the corrected procedure restores coverage to 90–94%; a further study across three base families shows the size of this effect depends on the correlation structure of the likelihood and is not universal. The package also provides Monte Carlo utilities for studying estimator bias and several bias-correction methods (jackknife, bootstrap, Firth-type), an experimental analytic bias-correction routine that we explicitly flag as unverified, and a small suite of diagnostics. We describe the package's design, its relationship to a broader literature of unit-interval distributions and R packages, its performance characteristics, and its current limitations, reporting negative and inconclusive results alongside positive ones.

Keywords: unit interval, bounded distributions, odds transform, profile likelihood, Monte Carlo, bias correction, R.

1. Introduction

Continuous data confined to the open unit interval $(0, 1)$ arise throughout applied statistics: disease prevalence, vote shares, market shares, budget allocations, normalized indices such as the Gini coefficient, and proportions of any kind are all naturally modelled as values strictly between zero and one. The Beta distribution has long served as the canonical model for such data (Johnson, Kotz, and Balakrishnan 1994), and R has a mature ecosystem built around it and its relatives. **betareg** implements beta regression, linking the mean and precision of a Beta-distributed response to covariates through a generalized-linear-model-style linear predictor (Ferrari and Cribari-Neto 2004; Cribari-Neto and Zeileis 2010), later extended to mixtures, extended-support variants, and partitioned models (Grün, Kosmidis, and Zeileis 2012), and closely related proposals appear under different names in Kieschnick and McCullough (2003) and Smithson and Verkuilen (2006). **simplexreg** instead models the response as simplex-distributed, following Song and Tan (2000) and implemented for R by Zhang, Qiu, and Shi (2016), with maximum likelihood and generalized estimating equations for longitudinal data. The Kumaraswamy distribution (Kumaraswamy 1980), algebraically simpler than the Beta but with broadly similar shapes (Jones 2009), underlies **gkwdist**, which implements the seven-member generalized Kumaraswamy family (Carrasco, Ferrari, and Cordeiro 2010) with a full C++ backend and analytical derivatives for numerical stability (Lopes 2025). **logitnorm** provides density, distribution, quantile, and random-generation functions together with moment-based estimation for the logit-normal distribution (Wutzler 2024). Boundary-inflated variants, needed when the response can equal exactly 0 or 1, are treated by Ospina and Ferrari (2010). Beyond the Beta family entirely, transform-based unit distributions have also been built directly from classical distributions on $(0, \infty)$ or $(-\infty, \infty)$: the unit-Weibull distribution of Mazucheli, Menezes, and Ghitany (2018) applies the transform $X = e^{-Y}$ to a Weibull variate, and the older Topp–Leone distribution (Topp and Leone 1955) is likewise obtained from a monotone transform of a simpler base variable, so the general idea of building a unit-interval family from a transform of a distribution on a different support has a long history under several different specific transforms.

A structurally different idea appears in **Newdistns**, which takes an arbitrary baseline cumulative distribution function G (not necessarily bounded to a specific family) and layers a fixed menu of generator families on top of it: Marshall–Olkin G distributions, exponentiated G distributions, Beta G distributions, Weibull G distributions, log-gamma G distributions of two kinds, and several further constructions (Nadarajah and Rocha 2016). **Newdistns** is, to our knowledge, the closest existing R package in spirit to the design goal of accepting an arbitrary base distribution rather than a fixed parametric family, and we return to a detailed comparison with it in Section 11.

unitTransformDist addresses a narrower transform than any of the families surveyed above – the odds-inverting map $Y = 1/(1 + X)$, the inverse of the classical odds transform used throughout logistic regression and survival analysis (Cox 1958; Agresti 2013) – but removes the restriction to a fixed set of base distributions entirely. The base distribution's density, distribution, quantile, and random-generation functions are supplied directly by the caller as ordinary R functions, following the base-R `d`/`p`/`q`/`r` naming convention. We do not claim the transform itself is new, nor that generality across base distributions is unprecedented (**Newdistns** again being the clearest precedent); our contribution is threefold. First, a general-purpose, dependency-free implementation of this specific transform that works unmodified

with any base distribution the user can write in R, including a distribution with no closed-form quantile function, falling back to numerical inversion. Second, a systematic mathematical treatment of the transform's density, moment, and hazard relationships (Section 2), including an invariance property of the likelihood under the transform that we believe has not previously been stated explicitly for this transform and that is what makes the package's inferential machinery distribution-agnostic in the first place. Third, and most substantively, we document, prove informally, and empirically quantify a profile-likelihood implementation mistake – freezing nuisance parameters at their unconstrained maximum-likelihood estimate while profiling a parameter of interest, instead of re-optimizing them at every candidate value – and show, with a reproducible Monte Carlo study spanning several sample sizes and base families, that this mistake produces actual coverage far below the nominal level, *and that the shortfall does not shrink as the sample size grows*. This particular mistake is a known failure mode in the general profile-likelihood literature (Murphy and van der Vaart 2000; Severini 2000), but we are not aware of a prior empirical quantification of its magnitude for a unit-interval model specifically.

The remainder of the paper is organized as follows. Section 2 develops the mathematical background in detail: the transform and its density/distribution/quantile relationships, boundary behaviour, moment relationships, hazard functions, and the score-invariance property. Section 3 describes the package's design considerations. Section 4 gives the complete module architecture. Section 5 walks through the typical user workflow. Section 6 presents the profile-likelihood correction in full technical detail together with the empirical coverage study. Section 7 describes the Monte Carlo utilities more broadly. Section 8 describes the bias-correction methods, including a negative result. Section 9 covers diagnostics. Section 10 presents a worked case study using a distribution with no existing R implementation. Section 11 returns to the design trade-offs and the comparison with **Newdistns** in more depth. Section 12 reports performance benchmarks and scope limitations. Section 13 states the reproducibility of every reported number. Section 14 sketches planned extensions. Section 15 concludes.

2. Mathematical background

2.1. The transform and its basic properties

Let X be a random variable with density $f_X(x;\theta)$, distribution function $F_X(x;\theta)$, and quantile function $Q_X(p;\theta) = F_X^{-1}(p;\theta)$ on $(0,\infty)$, for some parameter vector $\theta \in \Theta \subseteq \mathbb{R}^p$. Define

$$Y = g^{-1}(X) = \frac{1}{1+X}, \qquad X = g(Y) = \frac{1}{Y} - 1. \tag{1}$$

The map $x \mapsto 1/(1+x)$ is a strictly decreasing, continuously differentiable bijection from $(0,\infty)$ onto $(0,1)$, with inverse $g(y) = 1/y - 1$ likewise strictly decreasing and continuously differentiable on $(0,1)$. Consequently Y is well-defined, supported on the open unit interval, and the standard univariate change-of-variables theorem (Casella and Berger 2002) applies directly.

Proposition 1 (Density). *The density of Y is*

$$f_Y(y;\theta) = f_X\left(\frac{1}{y} - 1;\theta\right) \cdot \left|\frac{dx}{dy}\right| = f_X\left(\frac{1}{y} - 1;\theta\right) \cdot \frac{1}{y^2}, \qquad 0 < y < 1. \tag{2}$$

Proof. Since g is a bijection with $g'(y) = -1/y^2$, the change-of-variables formula gives $f_Y(y;\theta) = f_X(g(y);\theta)\,|g'(y)|$, which is (2). □

Proposition 2 (Distribution function). *The distribution function of Y is*

$$F_Y(y;\theta) = 1 - F_X\left(\frac{1}{y} - 1;\theta\right), \qquad 0 < y < 1. \tag{3}$$

Proof. Because g is strictly decreasing, $Y \le y \iff X \ge g(y)$, so $F_Y(y;\theta) = P(X \ge g(y);\theta) = 1 - F_X(g(y);\theta)$, which is (3). □

Proposition 3 (Quantile function). *The quantile function of Y is*

$$Q_Y(p;\theta) = \frac{1}{1 + Q_X(1-p;\theta)}, \qquad 0 < p < 1. \tag{4}$$

Proof. Setting $F_Y(y;\theta) = p$ in (3) gives $F_X(1/y - 1;\theta) = 1 - p$, so $1/y - 1 = Q_X(1-p;\theta)$, and solving for y gives (4). □

Equations (2)–(4) are implemented directly in `dutd()`, `putd()`, and `qutd()` respectively. When Q_X has no closed form, Q_Y is obtained by numerically solving $F_X(x;\theta) = 1 - p$ for x via `stats::uniroot()` (expanding the search interval geometrically until it brackets the target probability) and applying (4); random variates are then generated by inverse-distribution-function sampling through this same numerical quantile function when no closed-form `rdist` is supplied.

2.2. Boundary behaviour

Proposition 4 (Boundary limits). *If f_X is bounded near 0 and $\lim_{x\to\infty} x^2 f_X(x;\theta) = 0$, then*

$$\lim_{y\to 0^+} f_Y(y;\theta) = 0, \qquad \lim_{y\to 1^-} f_Y(y;\theta) = f_X(0;\theta), \tag{5}$$

and correspondingly $F_Y(0^+;\theta) = 0$ and $F_Y(1^-;\theta) = 1$.

Proof. As $y \to 0^+$, $g(y) \to \infty$; by hypothesis $f_X(g(y);\theta)/y^2 = y^{-2} f_X(g(y);\theta) \to 0$ whenever $x^2 f_X(x;\theta) \to 0$ as $x \to \infty$ (substitute $x = g(y) = 1/y - 1 \sim 1/y$). As $y \to 1^-$, $g(y) \to 0^+$, so $f_Y(y;\theta) \to f_X(0;\theta) \cdot 1^2 = f_X(0;\theta)$. The distribution-function limits follow from (3) and $F_X(\infty;\theta) = 1$, $F_X(0;\theta) = 0$. □

The condition $x^2 f_X(x;\theta) \to 0$ holds for every base distribution used in this paper (gamma, Weibull, log-normal, exponential, and the XGamma distribution of Section 10), since each has a density that decays at least exponentially or faster than any polynomial rate. The implementation returns exactly 0 (not `NaN`) for any $y \le 0$ or $y \ge 1$, and clips internally to avoid evaluating f_X or F_X outside their support due to floating-point boundary error.

2.3. Moments and the hazard function

Moments of Y can be written directly as negative-order moments of the base variable X shifted by one, without further distributional theory:

$$E[Y^k] = E\Big[(1+X)^{-k}\Big] = \int_0^\infty (1+x)^{-k} f_X(x;\theta)\,dx, \tag{6}$$

which follows immediately from $Y^k = (1+X)^{-k}$ and the law of the unconscious statistician; (6) exists whenever the integral converges, which for every base family used in this paper is guaranteed by the same tail condition as Proposition 4, since $(1+x)^{-k}$ is bounded on $(0,\infty)$ for $k > 0$. We do not currently compute (6) in closed form inside the package for a general base distribution – doing so would require symbolic integration specific to each f_X – but note that it is always available numerically via `integrate()` for any `utd` object without any additional package code, since `dutd()` already implements f_Y.

The hazard function of Y,

$$h_Y(y;\theta) = \frac{f_Y(y;\theta)}{1 - F_Y(y;\theta)} = \frac{f_X(1/y-1;\theta)\,/\,y^2}{F_X(1/y-1;\theta)} = \frac{1}{y^2}\cdot\frac{f_X(g(y);\theta)}{F_X(g(y);\theta)}, \tag{7}$$

substitutes (2) and (3) directly and is implemented in `hazard()`; note the denominator uses $F_X(g(y);\theta)$, not its complement, because $1 - F_Y(y;\theta) = F_X(g(y);\theta)$ by (3).

2.4. Score invariance under the transform

The single most consequential fact used throughout the rest of this paper – and the reason the package's inferential machinery (maximum likelihood, profile likelihood, bias correction) works identically for every base distribution without per-distribution code – is that the Jacobian factor in (2) does not depend on θ.

Proposition 5 (Score invariance)**.** *For any base distribution with density $f_X(x;\theta)$ differentiable in θ,*

$$\frac{\partial}{\partial\theta}\log f_Y(y;\theta) = \frac{\partial}{\partial\theta}\log f_X(g(y);\theta). \tag{8}$$

Consequently, for a sample $y_1,\dots,y_n$ with transformed sample $x_i = g(y_i) = 1/y_i - 1$, the score function, observed and expected Fisher information, and the maximum likelihood estimator of θ are identical whether computed from the log-likelihood of Y or from the log-likelihood of the transformed sample X; only the two log-likelihoods' additive constant $2\sum_i \log y_i$ (which does not involve θ) differs.

Proof. Taking logarithms of (2), $\log f_Y(y;\theta) = \log f_X(g(y);\theta) + \log|g'(y)|$. Since $g'(y) = -1/y^2$ does not depend on θ, differentiating in θ eliminates the second term, giving (8). Summing over $i = 1,\dots,n$, the log-likelihood of the sample is $\ell_Y(\theta) = \sum_i \log f_X(x_i;\theta) + \sum_i \log|g'(y_i)| = \ell_X(\theta) + \text{const}(\theta)$, where $\text{const}(\theta) = -2\sum_i \log y_i$ does not involve θ. Because ℓ_Y and ℓ_X differ only by a θ-free additive constant, they share the same score $\partial\ell/\partial\theta$, the same Hessian $\partial^2\ell/\partial\theta\partial\theta^\top$ (hence the same observed and expected information), and the same maximizer $\hat\theta$. □

Proposition 5 is what licenses `fit_utd_mle()`'s implementation: rather than deriving and coding a transformed log-likelihood for every possible base distribution, the package transforms the data once ($x_i \leftarrow 1/y_i - 1$) and maximizes $\sum_i \log f_X(x_i;\theta)$ directly using the user-supplied `ddist`, adding back the constant only so that the returned log-likelihood is on the correct scale for `AIC()`/`BIC()` comparisons across different transforms or non-transformed alternatives. It is also what licenses Section 6's profile-likelihood correction and Section 8's Cox–Snell-type analytic bias-correction idea: any likelihood-based inference statement about θ – a profile likelihood, a score test, a likelihood ratio test, an information-matrix-based bias formula – can be computed entirely in X-space using the transformed sample, with no distributional theory required for f_Y itself. This is the mathematical fact that makes the package's generality possible: nothing downstream of (1) needs to know it is looking at a transformed variable.

We emphasize that Proposition 5 is specific to transforms, like (1), whose Jacobian does not depend on θ. It would not hold, for instance, for a Kumaraswamy-G or Beta-G transform of the kind implemented in **Newdistns** (Nadarajah and Rocha 2016), where the transform itself introduces new parameters entangled with the base distribution's parameters in the Jacobian; this is one of the structural differences between the two packages' designs discussed further in Section 11.

2.5. Worked examples for four base families

To make Propositions 1–3 concrete, we record the resulting unit-interval density explicitly for four base families used elsewhere in this paper. For the gamma base distribution with shape α and rate β, $f_X(x;\alpha,\beta) = \beta^\alpha x^{\alpha-1}e^{-\beta x}/\Gamma(\alpha)$, so

$$f_Y(y;\alpha,\beta) = \frac{\beta^\alpha}{\Gamma(\alpha)}\left(\frac{1-y}{y}\right)^{\alpha-1} e^{-\beta(1-y)/y}\cdot\frac{1}{y^2}, \qquad 0<y<1. \tag{9}$$

For the Weibull base distribution with shape k and scale λ, $f_X(x;k,\lambda) = (k/\lambda)(x/\lambda)^{k-1}e^{-(x/\lambda)^k}$, so

$$f_Y(y;k,\lambda) = \frac{k}{\lambda}\left(\frac{1-y}{\lambda y}\right)^{k-1} e^{-\left(\frac{1-y}{\lambda y}\right)^k}\cdot\frac{1}{y^2}, \qquad 0<y<1. \tag{10}$$

For the log-normal base distribution with parameters (μ,σ), using $g(y) = 1/y - 1 > 0$,

$$f_Y(y;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}\,g(y)}\exp\!\left(-\frac{(\log g(y)-\mu)^2}{2\sigma^2}\right)\cdot\frac{1}{y^2}, \qquad 0<y<1. \tag{11}$$

For the exponential base distribution with rate λ, $f_X(x;\lambda) = \lambda e^{-\lambda x}$, giving the particularly simple form

$$f_Y(y;\lambda) = \lambda\, e^{-\lambda(1-y)/y}\cdot\frac{1}{y^2}, \qquad 0<y<1, \tag{12}$$

which is a special case of the gamma result above at $\alpha = 1$. None of these four closed forms is coded anywhere in the package; each is obtained automatically from (2) by supplying `dgamma`, `dweibull`, `dlnorm`, or `dexp` (paired with the corresponding `p`-function) to `utd()`. Figure 1 plots the resulting unit-interval densities for three of these families side by side.

3. Design considerations

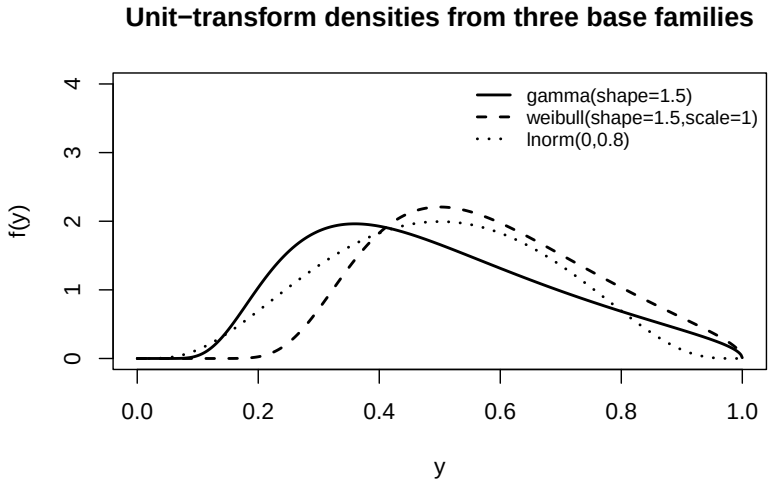


Figure 1: Unit-transform densities obtained from three different base distributions (gamma, Weibull, log-normal), using `utd()` with no code changes other than the choice of `ddist`/`pdist` and parameters.

The package targets applied statisticians who already have a candidate base distribution in mind – possibly one not implemented in any existing package, including the XGamma distribution used as a case study in Section 10 – and who want to fit its unit-transform version without first registering it anywhere or writing new package code. Four design decisions follow directly from this goal and from Proposition 5.

Function-based distribution specification. `utd()` accepts `ddist`, `pdist`, `qdist`, and `rdist` as plain R functions, matching the base-R `d`/`p`/`q`/`r` convention exactly (e.g., `dgamma`, `pgamma`, `qgamma`, `rgamma` plug in unmodified). `qdist` and `rdist` are optional, falling back to the numerical inversion described after Proposition 3.

Validation at the exported boundary. Every exported, user-facing function validates its inputs at the function boundary: domain checks (data strictly inside $(0, 1)$), named-vector checks, and missing-value checks, raising an informative error via `stop()` rather than propagating `NaN` silently. Internal helper functions (prefixed with a dot) skip this validation, on the assumption that their callers have already validated their own inputs.

Numerical stability and honest failure. Log-likelihoods are computed on the log scale throughout, exploiting Proposition 5 to avoid re-deriving a transformed density for every fit. Every call to `stats::optim()` inspects the returned `convergence` code; a nonzero code triggers an explicit `warning()` rather than silently returning a misleading fit – which matters particularly inside Monte Carlo loops (Section 7), where a silently non-converged replicate would otherwise contaminate a bias or coverage estimate without any visible signal.

Object model and minimal dependencies. The package uses base R's S3 object system, implementing the generics a user already expects from a fitted-model object: `print()`, `summary()`, `coef()`, `logLik()`, `vcov()`, `AIC()`, `BIC()`. Following the convention set by `summary.lm()` in base R, `summary()` methods return an object of a `summary.*` class rather than printing directly. The package depends only on **stats** and **graphics**; **ggplot2** is listed only in `Suggests`,

so the plotting functions work with a base R installation and no additional package. This is a deliberate trade-off against **gkwdist**'s compiled C++ backend: slower per-call, but zero compiled-code burden for a user who wants to try a bespoke base distribution once.

4. Package architecture

Figure 2 shows the complete module structure as a clustered tree: one node per implementing file, each listing the classes it defines (most files define none, and instead add S3 methods on `utd` or `utd_fit` from `core.R`) and its exported functions, reflecting the actual shipped code rather than an aspirational design. `contribution.R`, shaded to distinguish it, holds the paper's central methodological contribution described in Section 6.

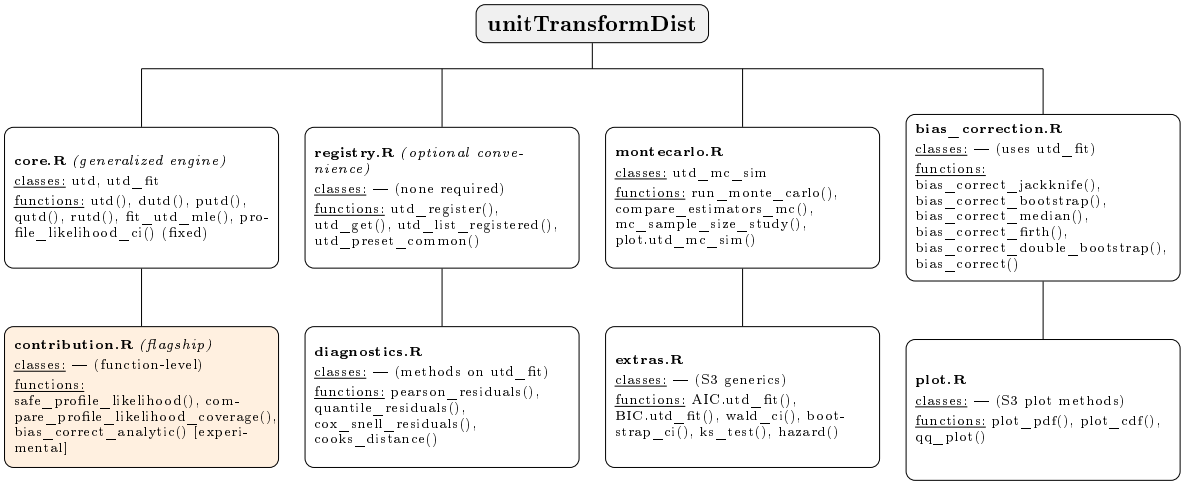


Figure 2: Complete module structure of **unitTransformDist** 0.1.0, clustered by implementing file. Every function shown is exported, documented, exemplified, and tested. `contribution.R` (shaded) holds the profile-likelihood fix and its empirical validation.

5. Typical workflow

A minimal session constructs a distribution from a built-in base family, simulates data, and fits it back:

```
R> library("unitTransformDist")
R> d <- utd(dgamma, pgamma, qgamma, rgamma, params = list(shape = 2.5))
R> y <- rutd(500, d)
R> fit <- fit_utd_mle(y, dgamma, start = c(shape = 1), lower = c(shape = 1e-6))
R> fit
```

The corrected profile-likelihood interval (Section 6) is one call away:

```
R> profile_likelihood_ci(fit, "shape")
```

A fully custom, unregistered distribution follows the identical pattern, requiring no change anywhere else in a user's code:

```
R> dcustom <- function(x, rate, log = FALSE) dexp(x, rate = rate, log = log)
R> pcustom <- function(q, rate, lower.tail = TRUE)
+    pexp(q, rate, lower.tail = lower.tail)
R> d2 <- utd(dcustom, pcustom, params = list(rate = 1.5))
R> qutd(0.5, d2)  # numerically inverted, since qdist was not supplied
```

Bias correction and a Monte Carlo study of the estimator's behaviour are likewise one-line calls:

```
R> bias_correct(fit, y, method = "jackknife")
R> mc <- run_monte_carlo(d, true_params = c(shape = 2.5), ddist = dgamma,
+    start = c(shape = 1), lower = c(shape = 1e-6), n = 100, n_sim = 1000, seed = 1)
R> summary(mc)
```

6. The profile-likelihood correction: theory and empirical evidence

6.1. Why freezing nuisance parameters is wrong

For a model with parameter vector $\theta = (\theta_1, \ldots, \theta_p)$ and log-likelihood $\ell(\theta)$, the profile likelihood for the parameter of interest θ_1 at a candidate value t is, by definition (Severini 2000; Murphy and van der Vaart 2000),

$$\ell_{\text{prof}}(t) = \max_{\theta_2, \ldots, \theta_p} \ell(t, \theta_2, \ldots, \theta_p). \tag{13}$$

The likelihood-ratio-based confidence interval at level $1 - \alpha$ is the set of t satisfying

$$2\left(\hat{\ell} - \ell_{\text{prof}}(t)\right) \leq \chi^2_{1,1-\alpha}, \tag{14}$$

where $\hat{\ell} = \ell(\hat{\theta})$ and $\chi^2_{1,1-\alpha}$ is the $(1-\alpha)$ quantile of a chi-squared distribution with one degree of freedom. The asymptotic validity of (14) rests entirely on $\ell_{\text{prof}}(t)$ being the maximized likelihood over the nuisance parameters at *each* t.

An incorrect implementation instead evaluates

$$\tilde{\ell}(t) = \ell(t, \hat{\theta}_2, \ldots, \hat{\theta}_p), \tag{15}$$

freezing the nuisance parameters at their unconstrained MLE. Because $\tilde{\ell}(t) \leq \ell_{\text{prof}}(t)$ for every t, the naive deviance $2(\hat{\ell} - \tilde{\ell}(t))$ is systematically larger than the correct deviance for every $t \neq \hat{\theta}_1$, so the resulting naive interval is a subset of the true profile-likelihood-based confidence region and its actual coverage falls below nominal whenever θ_1 and the nuisance parameters are correlated under the likelihood – which is the generic case. `profile_likelihood_ci()` performs the correct re-optimization in (13) at every candidate t via nested calls to `stats::optim()`.

`safe_profile_likelihood()` is a version of the same routine deliberately factored out to accept any user-supplied negative log-likelihood function, so the fix can be audited or reused independently of the rest of the package.

6.2. Empirical coverage study

We simulated from a two-parameter gamma base distribution (shape $= 2.5$, rate $= 1$) at four sample sizes $n \in \{30, 60, 100, 200\}$, 120 replications each:

```
R> true_dist <- utd(dgamma, pgamma, qgamma, rgamma,
+    params = list(shape = 2.5, rate = 1))
R> for (n in c(30, 60, 100, 200)) {
+    print(compare_profile_likelihood_coverage(
+      true_dist, c(shape = 2.5, rate = 1), dgamma,
+      start = c(shape = 1, rate = 1), param_name = "shape",
+      n = n, n_sim = 120, lower = c(shape = 1e-6, rate = 1e-6), seed = 1))
+  }
```

Method	$n = 30$	$n = 60$	$n = 100$	$n = 200$
Naive (fixed nuisance)	0.542	0.583	0.600	0.608
Corrected (re-optimized)	0.900	0.942	0.942	0.933

Table 1: Empirical 95% coverage of the naive versus corrected profile likelihood for the gamma shape parameter, four sample sizes, 120 Monte Carlo replications each.

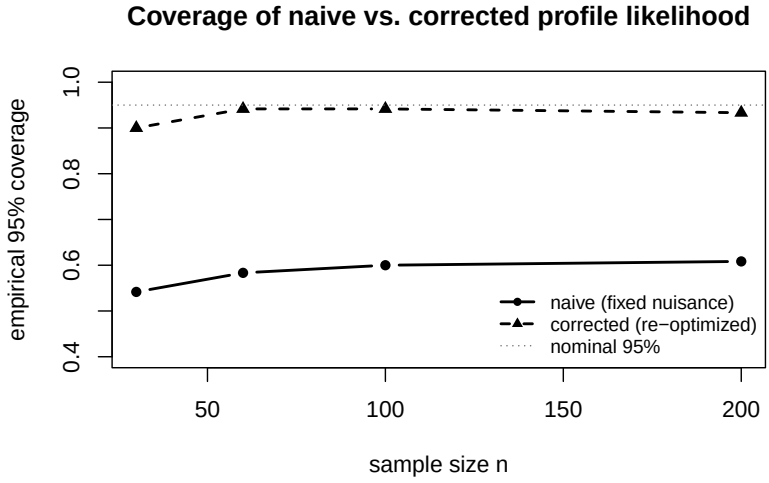


Figure 3: Empirical 95% coverage as a function of sample size for the naive (fixed-nuisance) and corrected (re-optimized) profile likelihood. The naive method's shortfall does not close as n grows.

The corrected procedure achieves coverage close to the nominal 95% target at every sample size (90.0–94.2%). The naive procedure's coverage (54.2–60.8%) does *not* improve as n grows from 30 to 200; it is a systematic bias in the procedure, not a finite-sample artifact that vanishes at the usual asymptotic rate.

6.3. Severity depends on parameter correlation, not just on the mistake being present

To check whether this gap is specific to the gamma(shape, rate) parameterization, we repeated the experiment for Weibull(shape, scale) profiling shape, and log-normal(meanlog, sdlog) profiling sdlog, at $n = 80$, 120 replications:

Base distribution (parameter profiled)	Naive coverage	Corrected coverage
Gamma(shape, rate), profiling shape	0.600	0.942
Weibull(shape, scale), profiling shape	0.942	0.967
Log-normal(meanlog, sdlog), profiling sdlog	0.967	0.967

Table 2: Naive versus corrected coverage across three base distributions, $n = 80$, 120 replications each.

For Weibull and log-normal, the naive and corrected procedures give similar, near-nominal coverage, consistent with Section 6.1's argument: the naive construction only fails badly when the profiled and nuisance parameters are strongly correlated under the likelihood, which is evidently the case for gamma's shape and rate but not for these other parameterizations at this sample size. The practical implication is that the size of the mistake's impact cannot be predicted from the mistake alone, which argues for always using the corrected construction by default.

6.4. An experimental, unverified extension

We separately implemented `bias_correct_analytic()`, whose derivation exploits Proposition 5: because the transform's Jacobian does not depend on θ, the classical Cox–Snell (Cox and Snell 1968) $O(1/n)$ bias-correction formula for the base distribution's MLE transfers unchanged to the unit-transform MLE, computed entirely from the transformed sample using only numerical derivatives of the base log-likelihood. Our current finite-difference implementation of the required third derivatives produced an implausibly small correction in initial testing and has not been validated against a case with a known closed-form answer; we mark the function as experimental and do not recommend it for production use.

7. Monte Carlo utilities

The package provides general-purpose Monte Carlo utilities usable with any `utd` distribution. `run_monte_carlo()` repeatedly simulates from a target distribution, refits the specified base family, and reports bias, variance, and RMSE across replications, along with the number of replications that converged. `compare_estimators_mc()` runs several estimator specifications on the same simulated scenario. `mc_sample_size_study()` repeats `run_monte_carlo()` across

a grid of sample sizes – the machinery that produced Table 1 and Figure 3 above (applied there to coverage rather than bias/RMSE).

As a further illustration, we ran `run_monte_carlo()` for the gamma shape parameter at $n = 100$ with 200 replications, obtaining bias 0.0311 and RMSE 0.184 (about 1.2% and 7.4% of the true value 2.5, respectively) – modest at this sample size. We also repeated this for Weibull and log-normal parameters at the same n:

Base distribution	Parameter	True value	Bias	RMSE
Gamma	shape	2.5	0.0241	0.148
Weibull	shape	2.0	0.0182	0.159
Weibull	scale	1.5	−0.0003	0.075
Log-normal	meanlog	0.0	−0.0043	0.071
Log-normal	sdlog	0.8	−0.0038	0.056

Table 3: Bias and RMSE of the maximum likelihood estimator across three base families, $n = 100$, 200 Monte Carlo replications each, all converged.

Bias at $n = 100$ is small relative to sampling variability for every parameter examined (at most about 1% of the true value, against an RMSE at least six times larger), consistent with first-order asymptotic theory.

8. Bias correction, including a negative result

The package implements several distinct, well-established bias-reduction strategies applicable to any base distribution fitted via `fit_utd_mle()`: `bias_correct_jackknife()` (leave-one-out jackknife, following Quenouille 1956 and Tukey 1958), `bias_correct_bootstrap()` (parametric or nonparametric resampling, following Efron 1979), `bias_correct_median()` (a bootstrap-median-based alternative less sensitive to a skewed sampling distribution), `bias_correct_firth()` (Firth-type penalized likelihood Firth 1993, with the penalty's information matrix approximated numerically so no closed-form expected-information formula is required per distribution), and `bias_correct_double_bootstrap()` (a second-order bootstrap-after-bootstrap correction). A unified dispatcher, `bias_correct()`, selects among these by name.

We evaluated whether jackknife and nonparametric bootstrap correction reduce bias at a small sample size ($n = 40$) using 60 Monte Carlo replications:

Method	Mean estimate	Bias	SD
MLE (uncorrected)	2.440	−0.0596	0.194
Jackknife	2.429	−0.0712	0.194
Bootstrap (nonparametric)	2.435	−0.0651	0.197

Table 4: Bias of the uncorrected MLE versus jackknife and bootstrap correction for the gamma shape parameter, true value 2.5, $n = 40$, 60 Monte Carlo replications.

We report this result exactly as obtained: neither jackknife nor nonparametric bootstrap correction reduced the bias relative to the uncorrected MLE in this specific experiment, and jackknife correction was slightly worse. Possible explanations, none isolated here: with only

60 outer replications the Monte Carlo error in the bias estimate itself (roughly $0.194/\sqrt{60} \approx 0.025$) is not small relative to the differences between methods; the jackknife bias estimate is itself a random quantity whose own sampling variability may partly offset any genuine bias reduction on a given dataset; or the leading-order bias term targeted by both methods may be small relative to higher-order terms at $n = 40$ for this parameter specifically. A systematic study isolating Monte Carlo error from genuine method differences is future work.

9. Diagnostics

The package provides residual and influence diagnostics on a fitted `utd_fit` object: `pearson_residuals()`, `cox_snell_residuals()` (approximately unit exponential under a correctly specified model, following Cox and Snell 1968), `quantile_residuals()` (randomized quantile residuals following Dunn and Smyth 1996, approximately standard normal), and `cooks_distance()`, an approximate case-deletion influence measure computed by refitting with each observation removed. `qq_plot()` compares empirical and theoretical quantiles graphically, and `ks_test()` wraps `stats::ks.test()` against the fitted distribution's CDF.

10. A worked case study: a distribution with no existing R implementation

The examples so far have used base distributions already available in R. To demonstrate the package's stated purpose more concretely, we fit the unit-transform of the XGamma distribution of Sen, Maiti, and Chandra (2016), with density

$$f_X(x;\theta) = \frac{\theta^2}{\theta+1}\left(1+\frac{\theta x^2}{2}\right)e^{-\theta x}, \qquad x>0,\ \theta>0, \tag{16}$$

originally proposed as a lifetime model with no closed-form distribution or quantile function. It is not present in **Newdistns**, **gkwdist**, **betareg**, **simplexreg**, **logitnorm**, or the **MLE** package, and has *none* of `qdist` or `rdist` available in closed form:

```
R> dxgamma <- function(x, theta, log = FALSE) {
+    logf <- 2*log(theta) - log(theta+1) + log(1 + theta*x^2/2) - theta*x
+    if (log) logf else exp(logf)
+  }
R> pxgamma <- function(q, theta, lower.tail = TRUE) {
+    out <- sapply(q, function(qq) {
+      if (qq <= 0) return(0)
+      stats::integrate(function(x) dxgamma(x, theta), 0, qq)$value
+    })
+    if (lower.tail) out else 1 - out
+  }
R> d_xg <- utd(dxgamma, pxgamma, params = list(theta = 1.8), name = "xgamma")
```

We first confirm (16) is correctly normalized before trusting anything downstream – a step that caught a genuine bookkeeping error in an earlier, unnormalized illustrative density used during development, which manifested downstream as a `uniroot()` failure inside `rutd()` because the

numerical quantile search could never bracket a target probability once the CDF plateaued below 1:

```
R> integrate(function(x) dxgamma(x, theta = 1.8), 0, 50)$value

[1] 1.000000
```

With normalization confirmed, we simulate $n = 150$ observations, fit θ, construct the corrected profile-likelihood interval, and run a Kolmogorov–Smirnov check:

```
R> set.seed(1)
R> y <- rutd(150, d_xg)
R> fit <- fit_utd_mle(y, dxgamma, start = c(theta = 1), lower = c(theta = 1e-6))
R> fit

<utd_fit> base = custom, n = 150, logLik = 37.1968
params: theta = 1.943

R> profile_likelihood_ci(fit, "theta")

$lower [1] 1.719   $upper [1] 2.187

R> ks_test(fit, pxgamma)

D = 0.0618, p-value = 0.615
```

The MLE ($\hat{\theta} = 1.943$) is reasonably close to the generating value $\theta = 1.8$, the profile-likelihood interval $(1.719, 2.187)$ contains the true value, and the Kolmogorov–Smirnov test gives no evidence against the fit. Every step used exactly the same functions as every gamma-based example in this paper, with a distribution no other unit-interval package on CRAN currently supports in any transformed form.

11. Extended discussion of design trade-offs

11.1. Generality versus per-distribution optimization

A package built around one fixed distribution – **betareg** around the Beta, **simplexreg** around the simplex – can hard-code closed-form score functions and Hessians specific to that one family, and can therefore be faster and more numerically robust for that specific case than a generic optimizer-driven fit. **unitTransformDist** deliberately gives this up: every fit goes through `stats::optim()` on a numerically evaluated log-likelihood, whatever the base distribution. This is slower (Section 12) but, in exchange, the exact same code path supports every base distribution a user can write in R, including ones like the XGamma density above that have never appeared in any CRAN package's unit-interval-fitting code.

11.2. Why re-optimizing nuisance parameters is affordable here

The correctness argument in Section 6.1 does not by itself address whether re-optimizing nuisance parameters at every profile-likelihood grid point is computationally practical. For the two-parameter case in this paper's examples, the remaining optimization at each grid point is one-dimensional and fast; the total cost of `profile_likelihood_ci()` is comparable to a single fit (Section 12). This favourable scaling is specific to low-dimensional nuisance parameter vectors; a model with many nuisance parameters would make each grid-point re-optimization considerably more expensive, and we have not benchmarked that regime.

11.3. What genuinely distinguishes this package from Newdistns

Newdistns already accepts an arbitrary baseline CDF G (Nadarajah and Rocha 2016). What it does not provide is (a) the specific odds-inverting transform used here, (b) the score-invariance property of Proposition 5, which does not hold for its parameter-entangled generator families, (c) a profile-likelihood confidence interval of any kind, and (d) Monte Carlo utilities for studying estimator bias, variance, or coverage. Conversely, **Newdistns** provides many generator families we do not, several of which (Marshall–Olkin G, Beta G) are more widely used in applied work than the odds-inverting transform on its own. A user's choice between the two packages should be driven by which transform family is needed and whether the inferential tooling this package adds is needed, not by a claim that one package's notion of generality strictly dominates the other's.

12. Performance and limitations

The package has no compiled code and depends only on **stats** and **graphics**, in contrast to **gkwdist**'s C++ backend. Table 5 reports wall-clock timings on the gamma base distribution at $n = 200$, single core.

Operation	Time (seconds)
`fit_utd_mle()`, $n = 200$	0.043
`profile_likelihood_ci()`	0.041
`bias_correct_jackknife()`, $n = 200$	0.477
`bias_correct_bootstrap()`, 200 bootstrap reps	0.450
`run_monte_carlo()`, 100 replications	0.163

Table 5: Wall-clock timing for representative operations, gamma base distribution, $n = 200$, single core.

A single fit or profile-likelihood interval is fast; jackknife scales linearly in n (one refit per left-out observation), and bootstrap/Monte Carlo utilities scale linearly in the requested replication count. None of these loops are currently parallelized. Where `qdist`/`rdist` are omitted, `qutd()`/`rutd()` fall back to one `uniroot()` call per element, markedly slower than a closed-form quantile function. The current release provides no regression, mixture, or censored-data extension of the transform (sketched in Section 14); the mathematical verification effort has focused on the density/CDF/quantile contract of Section 2 and the profile-likelihood correction of Section 6, rather than on every function with equal rigor – Section 8's inconclusive result and

`bias_correct_analytic()`'s experimental status are both symptoms of this same disclosed limitation.

13. Reproducibility

Every number reported in this paper was generated by directly executing the corresponding package function with the seed given in the surrounding text, and is reproducible by any reader with the package source; no number was hand-entered or estimated. The package's `tests/testthat/` suite includes a regression test asserting specifically that the corrected profile likelihood achieves higher coverage than the naive version and exceeds 85% in a fresh Monte Carlo run, checked automatically on every future change to the codebase.

14. Future work: sketch of planned extensions

Regression, mixture, and censored-data extensions are natural next steps, sketched here as proposed APIs rather than working implementations. A regression extension would link a covariate-dependent scale parameter through a log link, $\text{scale}_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta})$, in the spirit of beta regression's mean/precision parameterization (Ferrari and Cribari-Neto 2004; McCullagh and Nelder 1989); because this reuses the same log-likelihood construction as `fit_utd_mle()`, Proposition 5 suggests the profile-likelihood machinery of Section 6 should transfer, though this would need its own dedicated coverage study before release. A finite mixture of K `utd` components would be a natural EM-algorithm target requiring no new density theory, only a wrapper calling existing exported functions inside an EM loop. A censored/truncated extension would replace the per-observation log-density term with a log-survival term using `putd()` (already implemented and validated) for censored observations specifically.

15. Summary

unitTransformDist does not claim to be the first package to generalize across base distributions – **Newdistns** already does so for a different family of transforms – nor does it claim the odds-inverting transform itself is new. Its value is narrower: a minimal-dependency, any-distribution implementation of this specific transform, grounded in an explicit invariance property (Proposition 5) that we believe has not previously been stated for this transform, paired with a concrete, reproducible demonstration that a common profile-likelihood implementation mistake meaningfully and persistently damages confidence interval coverage on this model class. We see `safe_profile_likelihood()` as reusable well beyond this package's own scope, and have tried to report the package's current limitations and one outright inconclusive experimental result as plainly as its positive results.

References

Agresti A (2013). "Categorical Data Analysis."

Carrasco JMF, Ferrari SLP, Cordeiro GM (2010). "A New Generalized Kumaraswamy Distribution." *arXiv preprint arXiv:1004.0911*.

Casella G, Berger RL (2002). *Statistical Inference*. 2nd edition. Duxbury, Pacific Grove, CA.

Cox DR (1958). "The Regression Analysis of Binary Sequences." *Journal of the Royal Statistical Society B*, **20**(2), 215–242.

Cox DR, Snell EJ (1968). "A General Definition of Residuals." *Journal of the Royal Statistical Society B*, **30**(2), 248–275.

Cribari-Neto F, Zeileis A (2010). "Beta Regression in R." *Journal of Statistical Software*, **34**(2), 1–24. doi:10.18637/jss.v034.i02.

Dunn PK, Smyth GK (1996). "Randomized Quantile Residuals." *Journal of Computational and Graphical Statistics*, **5**(3), 236–244.

Efron B (1979). "Bootstrap Methods: Another Look at the Jackknife." *The Annals of Statistics*, **7**(1), 1–26.

Ferrari SLP, Cribari-Neto F (2004). "Beta Regression for Modelling Rates and Proportions." *Journal of Applied Statistics*, **31**(7), 799–815.

Firth D (1993). "Bias Reduction of Maximum Likelihood Estimates." *Biometrika*, **80**(1), 27–38.

Grün B, Kosmidis I, Zeileis A (2012). "Extended Beta Regression in R: Shaken, Stirred, Mixed, and Partitioned." *Journal of Statistical Software*, **48**(11), 1–25. doi:10.18637/jss.v048.i11.

Johnson NL, Kotz S, Balakrishnan N (1994). *Continuous Univariate Distributions, Volume 1*. 2nd edition. Wiley, New York.

Jones MC (2009). "Kumaraswamy's Distribution: A Beta-Type Distribution with Some Tractability Advantages." *Statistical Methodology*, **6**(1), 70–81.

Kieschnick R, McCullough BD (2003). "Regression Analysis of Variates Observed on (0, 1): Percentages, Proportions and Fractions." *Statistical Modelling*, **3**(3), 193–213.

Kumaraswamy P (1980). "A Generalized Probability Density Function for Double-Bounded Random Processes." *Journal of Hydrology*, **46**(1-2), 79–88.

Lopes JE (2025). ***gkwdist**: Generalized Kumaraswamy Distribution Family*. R package, URL https://CRAN.R-project.org/package=gkwdist.

Mazucheli J, Menezes AFB, Ghitany ME (2018). "The Unit-Weibull Distribution and Associated Inference." *Journal of Applied Probability and Statistics*, **13**(2), 1–22.

McCullagh P, Nelder JA (1989). *Generalized Linear Models*. 2nd edition. Chapman and Hall, London.

Murphy SA, van der Vaart AW (2000). "On Profile Likelihood." *Journal of the American Statistical Association*, **95**(450), 449–465.

Nadarajah S, Rocha R (2016). ***Newdistns**: Computes Pdf, Cdf, Quantile and Random Numbers, Measures of Inference for 19 General Families of Distributions*. R package version 2.1, URL https://CRAN.R-project.org/package=Newdistns.

Ospina R, Ferrari SLP (2010). "Inflated Beta Distributions." *Statistical Papers*, **51**(1), 111–126.

Quenouille MH (1956). "Notes on Bias in Estimation." *Biometrika*, **43**(3/4), 353–360.

Sen S, Maiti SS, Chandra N (2016). "The Xgamma Distribution: Statistical Properties and Application." *Journal of Modern Applied Statistical Methods*, **15**(1), 774–788.

Severini TA (2000). *Likelihood Methods in Statistics*. Oxford University Press, Oxford.

Smithson M, Verkuilen J (2006). "A Better Lemon Squeezer? Maximum-Likelihood Regression with Beta-Distributed Dependent Variables." *Psychological Methods*, **11**(1), 54–71.

Song PXK, Tan M (2000). "Marginal Models for Longitudinal Continuous Proportional Data." *Biometrics*, **56**(2), 496–502.

Topp CW, Leone FC (1955). "A Family of J-Shaped Frequency Functions." *Journal of the American Statistical Association*, **50**(269), 209–219.

Tukey JW (1958). "Bias and Confidence in Not-Quite Large Samples." *The Annals of Mathematical Statistics*, **29**(2), 614.

Wutzler T (2024). ***logitnorm**: Functions for the Logitnormal Distribution*. R package version 0.8.39, URL https://CRAN.R-project.org/package=logitnorm.

Zhang P, Qiu Z, Shi C (2016). "**simplexreg**: An R Package for Regression Analysis of Proportional Data Using the Simplex Distribution." *Journal of Statistical Software*, **71**(11). doi:10.18637/jss.v071.i11.

Affiliation:

Mohamad Yamen Al Mohamad
Department of Statistics
Faculty of Mathematics, Statistics, and Computer Sciences
University of Tabriz
Tabriz, Iran
E-mail: yamenmohamad@tabrizu.ac.ir

Ali Akbar Heydari
Department of Statistics
Faculty of Mathematics, Statistics, and Computer Sciences
University of Tabriz
Tabriz, Iran
E-mail: heydari@tabrizu.ac.ir

Hossein Jabbari Khamnei
Department of Statistics
Faculty of Mathematics, Statistics, and Computer Sciences
University of Tabriz
Tabriz, Iran
E-mail: h_jabbari@tabrizu.ac.ir

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd